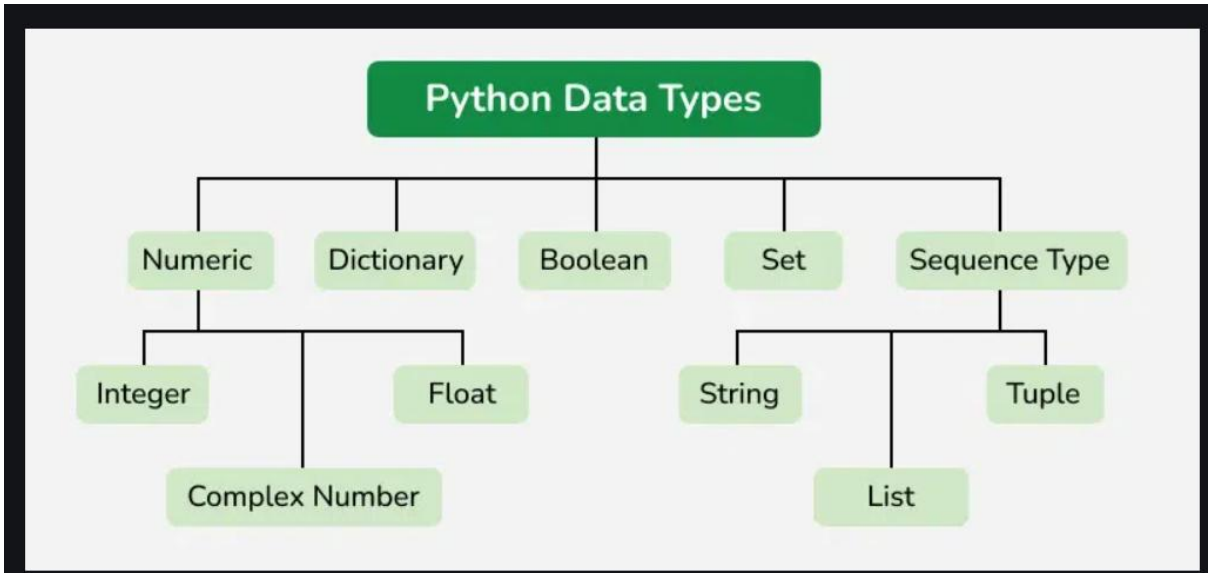
List diff data types in Python

Data types in Python are a way to classify data items. They represent the kind of value which determines what operations can be performed on that data. Since everything is an object in Python programming, Python data types are classes and variables are instances (objects) of these classes.

The following are standard or built-in data types in Python:

- Numeric: int, float, complex
- Sequence Type: string, list, tuple
- Mapping Type: dict
- Boolean: bool
- Set Type: set, frozenset
- Binary Types: bytes, bytearray, memoryview



1. Numeric Data Types

Python numbers represent data that has a numeric value. A numeric value can be an integer, a floating number or even a complex number. These values are defined as int, float and complex classes.

- Integers: value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). There is no limit to how long an integer value can be.
- Float: value is represented by float class. It is a real number with a floating-point representation. It is specified by a decimal point. Optionally, character e or E followed by a positive or negative integer may be appended to specify scientific notation.

- Complex Numbers: It is represented by a complex class. It is specified as (real part) + (imaginary part)j. For example - 2+3j

```
a = 5

print(type(a))


b = 5.0

print(type(b))


c = 2 + 4j

print(type(c))
```

Output

```
<class 'int'>

<class 'float'>

<class 'complex'>
```

2. Sequence Data Types

A sequence is an ordered collection of items, which can be of similar or different data types. Sequences allow storing of multiple values in an organized and efficient fashion. There are several sequence data types of Python:

String Data Type

Python Strings are arrays of bytes representing Unicode characters. In Python, there is no character data type, a character is a string of length one. It is represented by str class.

Strings in Python can be created using single quotes, double quotes or even triple quotes. We can access individual characters of a String using index.

```
s = 'Welcome to the Geeks World'

print(s)


# check data type

print(type(s))


# access string with index

print(s[1])
```

```
print(s[2])

print(s[-1]) # -1 refers to the last character, -2 is second last, and so on
```

Output

```
Welcome to the Geeks World

<class 'str'>

e

l

d
```

List Data Type

Lists are similar to arrays found in other languages. They are an ordered and mutable collection of items. It is very flexible as items in a list do not need to be of the same type.

Creating a List in Python: Lists can be created by just placing sequence inside the square brackets[].

```
# Empty list

a = []


# list with int values

a = [1, 2, 3]

print(a)


# list with mixed values int and String

b = ["Geeks", "For", "Geeks", 4, 5]

print(b)
```

Output

```
[1, 2, 3]

['Geeks', 'For', 'Geeks', 4, 5]
```

Access List Items: In order to access the list items refer to index number. In Python, negative sequence indexes represent positions from end of the array. Instead of having to compute offset as in List[len(List)-3], it is enough to just write List[-3]. Negative indexing means beginning from end, -1 refers to last item, -2 refers to second-last item, etc.

```
a = ["Geeks", "For", "Geeks"]
print("Accessing element from the list")
print(a[0])
print(a[2])

print("Accessing element using negative indexing")
print(a[-1])
print(a[-3])
```

Output

```
Accessing element from the list
Geeks
Geeks
Accessing element using negative indexing
Geeks
Geeks
```

Tuple Data Type

Tuple is an ordered collection of Python objects. The only difference between a tuple and a list is that tuples are immutable. Tuples cannot be modified after it is created.

Creating a Tuple in Python: tuples are created by placing a sequence of values separated by a 'comma' with or without the use of parentheses for grouping data sequence. Tuples can contain any number of elements and of any datatype (like strings, integers, lists, etc.).

Note: Tuples can also be created with a single element, but it is a bit tricky. Having one element in the parentheses is not sufficient, there must be a trailing 'comma' to make it a tuple.

```
# initiate empty tuple
tup1 = ()

tup2 = ('Geeks', 'For')
print("\nTuple with the use of String: ", tup2)
```

Output

```
Tuple with the use of String:  ('Geeks', 'For')
```

Note: The creation of a Python tuple without the use of parentheses is known as Tuple Packing.

Access Tuple Items: In order to access tuple items refer to the index number. Use the index operator [ ] to access an item in a tuple.

```
tup1 = (1, 2, 3, 4, 5)


# access tuple items

print(tup1[0])

print(tup1[-1])

print(tup1[-3])
```

Output

1

5

3

3. Boolean Data Type

Python Boolean Data type is one of the two built-in values, True or False. Boolean objects that are equal to True are truthy (true) and those equal to False are falsy (false). However non-Boolean objects can be evaluated in a Boolean context as well and determined to be true or false. It is denoted by class bool.

```
print(type(True))

print(type(False))

print(type(true))
```

Output:

<class 'bool'>
<class 'bool'>

Hangup (SIGHUP)
Traceback (most recent call last):
File "/home/guest/sandbox/Solution.py", line 3, in <module>
print(type(true))
*^^^^*
NameError: name 'true' is not defined. Did you mean: 'True'?

Truthy and Falsy Values

truthy and falsy values are values that evaluate to True or False in a Boolean context. Truthy values behave like True, while falsy values behave like False when used in conditions.

```
if 1:
```

```
    print("1 is truthy")


if not 0:
    print("0 is falsy")
```

Output

1 is truthy

0 is falsy

4. Set Data Type

Set is an unordered collection of data types that is iterable, mutable, and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

Create a Set in Python

Sets can be created by using the built-in set() function with an iterable object or a sequence by placing the sequence inside curly braces, separated by a 'comma'. The type of elements in a set need not be the same, various mixed-up data type values can also be passed to the set.

```
# initializing empty set
s1 = set()


s1 = set("GeeksForGeeks")
print("Set with the use of String: ", s1)


s2 = set(["Geeks", "For", "Geeks"])
print("Set with the use of List: ", s2)
```

Output

Set with the use of String: {'s', 'o', 'F', 'G', 'e', 'k', 'r'}

Set with the use of List: {'Geeks', 'For'}

Note: Sets in Python are unordered, so the order of elements when printed may vary each time the code is run.

Access Set Items

Set items cannot be accessed by referring to an index, since sets are unordered the items have no index. But we can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the keyword in.

```
set1 = set(["Geeks", "For", "Geeks"]) #Duplicates are removed automatically

print(set1)


# loop through set

for i in set1:

  print(i, end=" ") #prints elements one by one


# check if item exist in set

print("Geeks" in set1)
```

Output

{'For', 'Geeks'}

For Geeks True

5. Dictionary Data Type

A dictionary in Python is a collection of data values, used to store information like a map. Unlike other Python data types, a dictionary holds key-value pairs, where each key maps to a value, allowing efficient access and retrieval of data. Each key-value pair in a dictionary is separated by a colon (:), and multiple pairs are separated by commas.

Create a Dictionary in Python

Values in a dictionary can be of any datatype and can be duplicated, whereas keys can't be repeated and must be immutable. The dictionary can also be created by the built-in function dict().

Note: Dictionary keys are case sensitive, the same name but different cases of Key will be treated distinctly.

```
# initialize empty dictionary

d = {}


d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}

print(d)
```

```
# creating dictionary using dict() constructor
d1 = dict({1: 'Geeks', 2: 'For', 3: 'Geeks'})
print(d1)
```

Output

{1: 'Geeks', 2: 'For', 3: 'Geeks'}

{1: 'Geeks', 2: 'For', 3: 'Geeks'}

Accessing Key-value in Dictionary

In order to access items of a dictionary refer to its key name. Key can be used inside square brackets. Using get() method we can access dictionary elements.

```
d = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

# Accessing an element using key
print(d['name'])

# Accessing a element using get
print(d.get(3))
```

Output

For

Geeks

Explain diff access modifiers in Python

Access modifiers in Python control which parts of a class can be accessed from outside the class, from within the class, or by subclasses. They help control how data and methods are accessed and used.

Types of Access Modifiers

1. Public Access Modifier

- Members (variables or methods) declared as public can be accessed from anywhere in the program.

- By default, all members are public in Python.

```
class Geek:
    def __init__(self, name, age):
        self.geekName = name
        self.geekAge = age
    def displayAge(self):
        print("Age:", self.geekAge)

obj = Geek("R2J", 20)

print("Name:", obj.geekName)
obj.displayAge()
```

Output

Name: R2J

Age: 20

2. Protected Access Modifier

- A member is considered protected if its name starts with a single underscore (_).
- Convention only: It suggests that the member should not be accessed outside the class except by subclasses.
- Still, Python allows direct access if explicitly called.

```
class Student:
    def __init__(self, name, roll, branch):
        self._name = name
        self._roll = roll
        self._branch = branch

    def _displayRollAndBranch(self):
        print("Roll:", self._roll)
        print("Branch:", self._branch)
```

```
class Geek(Student):
    def displayDetails(self):
        print("Name:", self._name)
        self._displayRollAndBranch()



obj = Geek("R2J", 1706256, "IT")
obj.displayDetails()
```

Output

Name: R2J

Roll: 1706256

Branch: IT

3. Private Access Modifier

- A member is private if its name starts with double underscores (__).
- Python does not enforce strict privacy — instead, it uses Name Mangling.
- The interpreter renames __var to _ClassName__var internally.

Note: A name like __age__ (double underscores at both start and end) is not private. It is treated as a special 'dunder' name in Python and name mangling does not apply to it.

```
class Geek:
    def __init__(self, name, roll, branch):
        self.__name = name
        self.__roll = roll
        self.__branch = branch

    def __displayDetails(self):
        print("Name:", self.__name)
        print("Roll:", self.__roll)
        print("Branch:", self.__branch)
```

```
    def accessPrivateFunction(self):
        self.__displayDetails()



obj = Geek("R2J", 1706256, "CSE")
obj.accessPrivateFunction()
print(obj._Geek__name)
```

Output

Name: R2J

Roll: 1706256

Branch: CSE

R2J

What is module in Python? Explain how can you use module in your program in Python

A Module in python is a file containing definitions and statements. A module can define functions, classes and variables. Modules help organize code into separate files so that programs become easier to maintain and reuse. Instead of writing everything in one place, related functionality can be grouped into its own module and imported whenever needed.

Create a Module

To create a Python module, write the desired code and save that in a file with .py extension.

Example: Let's create a calc.py in which we define two functions, one add and another subtract.

```
# calc.py
def add(x, y):
    return (x+y)


def subtract(x, y):
    return (x-y)
```

This is all that is required to create a module.

Import module

Modules can be used in another Python file using the import statement. When Python sees an import, it loads the module if it exists in the interpreter's search path. Below is the syntax to import a module:

import module

Example: Here, we are importing the calc that we created earlier to perform add operation.

```
import calc

print(calc.add(10, 2))
```

Output

12

Explanation: import calc loads the module and calc.add() accesses a function through dot notation.

Types of Import Statements

1. Import From Module: This allows importing specific functions, classes, or variables rather than the whole module.

```
from math import sqrt, factorial

print(sqrt(16))

print(factorial(6))
```

Output

4.0

720

Explanation: Only sqrt and factorial are brought into the local namespace, so the prefix math. is not required.

2. Import All Names: * imports everything from a module into the current namespace.

```
from math import *

print(sqrt(16))

print(factorial(6))
```

Output

4.0

720

Explanation: Every public name of math becomes directly accessible. (Not recommended in large projects due to namespace conflicts.)

3. Import With Alias: You can shorten a module's name using as.

```
import math as m

print(m.pi)
```

Output

3.141592653589793

Explanation: math is accessed through the shorter alias m.

Types of Modules

Python provides several kinds of modules. Each type plays a different role in application development.

1. Built-in Modules: These come bundled with Python and require no installation - e.g., math, random, os.

```
import random

print(random.randint(1, 5))
```

Output

4

Explanation: random.randint() returns a random number within the given range.

2. User-Defined Modules: These are modules you create yourself, such as calc.py.

```
import calc

print(calc.sub(20, 5))
```

Output

15

Explanation: The module is created manually and then imported into another script.

3. External (Third-Party) Modules: These modules are installed using pip - e.g., NumPy, Pandas, Requests.

```
import requests

r = requests.get("https://example.com")

print(r.status_code)
```

Output

200

Explanation: requests is installed separately (pip install requests) and provides HTTP utilities.

4. Package Modules: A package is a directory containing multiple modules, usually with an __init__.py file.

Example Directory

mypkg/
__init__.py
calc.py
utils.py

Explain Python built in Exceptions.

Python provides a set of built-in exceptions, each designed to signal a specific type of error and help you debug more effectively.

These built-in exceptions can be viewed using the local() built-in functions as follows :

>>> locals()['__builtins__']

This returns a dictionary of built-in exceptions, functions and attributes.

Examples of Built in Exceptions

1. BaseException

The BaseException class is the root of Python's exception hierarchy. All other exceptions directly or indirectly inherit from it. While it is rarely used directly in code, it is important because it forms the foundation of Python’s error-handling system.

Example: This example manually raises a BaseException and catches it to show how the root exception works.

```
try:
    raise BaseException("This is a BaseException")
except BaseException as e:
    print(e)
```

Output

This is a BaseException

Explanation: Here, we forcefully raise a BaseException. Since we catch it in the except block, message is printed instead of the program crashing.

2. Exception

The Exception class is the base for all non-exit exceptions. You will often catch Exception in general error-handling code when you are not targeting a specific error type.

Example: This code raises a generic Exception and handles it inside the except block.

```
try:
    raise Exception("This is a generic exception")
except Exception as e:
    print(e)
```

Output

This is a generic exception

Explanation: We manually raise an Exception with a message. The error is caught and message is displayed instead of halting the program.

3. ArithmeticError

The ArithmeticError class is the base for all errors related to mathematical operations. You don't usually raise it directly, but it provides a way to catch all math-related errors in one block.

Example: This example raises an ArithmeticError manually to demonstrate how it works.

```
try:
    raise ArithmeticError("Arithmetic error occurred")
except ArithmeticError as e:
    print(e)
```

Output

Arithmetic error occurred

Explanation: We raise an ArithmeticError with a custom message. The program catches it and prints the error message, preventing a crash.

4. ZeroDivisionError

A ZeroDivisionError occurs when you attempt to divide a number by zero. Since division by zero is undefined in mathematics, Python raises this exception to signal the error.

Example: This code attempts to divide 10 by 0, which triggers a ZeroDivisionError.

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(e)
```

Output

```
division by zero
```

Explanation: The operation 10/0 is invalid. Python raises a ZeroDivisionError, which is then caught in the except block. Instead of crashing, program prints the error message.

5. OverflowError

An OverflowError occurs when the result of a numerical operation is too large for Python to represent. While Python handles large integers well, certain floating-point operations (like very large exponentials) can still cause this error.

Example: This example uses the math.exp() function with a very large input, which causes an overflow.

```
import math
try:
    result = math.exp(1000)  # Exponential function with a large argument
except OverflowError as e:
    print(e)
```

Output

```
math range error
```

Explanation: The exponential function with input 1000 produces a number too large to handle. Python raises an OverflowError, which is caught and displayed as an error message.

6. FloatingPointError

The FloatingPointError occurs when a floating-point calculation fails. By default, Python handles most floating-point issues silently (like dividing by zero results in inf or nan). However, you can explicitly enable floating-point error reporting with libraries like NumPy.

Example: This example enables error reporting in NumPy and performs a division by zero to trigger FloatingPointError.

```
import numpy as np
np.seterr(all='raise')
```

```
try:
    np.divide(1, 0)
except FloatingPointError as e:
    print("FloatingPointError caught:", e)
```

Output

FloatingPointError caught: divide by zero encountered in divide

Explanation: Normally, NumPy would return inf for 1/0. But since we enabled error raising, the division explicitly throws a FloatingPointError, which we catch and print.

7. AssertionError

An AssertionError is raised when the assert statement fails. The assert keyword is often used for debugging or testing assumptions in code.

Example: This example checks if 1 == 2 using assert. Since the condition is false, it raises an AssertionError.

```
try:
    assert 1 == 2, "Assertion failed"
except AssertionError as e:
    print(e)
```

Output

Assertion failed

Explanation: The assert statement expects 1 == 2 to be true, but it isn't. Therefore, Python raises an AssertionError with the message "Assertion failed".

8. AttributeError

An AttributeError occurs when you try to access or assign an attribute that does not exist for an object.

Example: This example tries to access a non-existent attribute in a class instance.

```
class MyClass:
    pass

obj = MyClass()
```

```
try:
    obj.some_attribute
except AttributeError as e:
    print(e)
```

Output

'MyClass' object has no attribute 'some_attribute'

Explanation: The object obj is created from MyClass, which has no attributes. Attempting to access some_attribute raises an AttributeError.

9. IndexError

An IndexError happens when you try to access a list (or any sequence) element with an index that is out of range.

Example: This example tries to access the 6th element of a list that only has 3 elements.

```
my_list = [1, 2, 3]

try:
    element = my_list[5]
except IndexError as e:
    print(e)
```

Output

list index out of range

Explanation: The list has only indices 0, 1, 2. When we try to access index 5, Python raises an IndexError.

10. KeyError

A KeyError occurs when you try to access a dictionary key that doesn't exist.

Example: This example tries to access the key "key2" in a dictionary that only has "key1".

```
d = {"key1": "value1"}

try:
```

```
    val = d["key2"]
except KeyError as e:
    print(e)
```

Output

'key2'

Explanation: The dictionary does not contain "key2". Attempting to retrieve it raises a KeyError, which is caught and printed.

11. MemoryError

A MemoryError occurs when Python cannot allocate enough memory for an operation. This usually happens when trying to create extremely large data structures.

Example: This example tries to create a very large list, which may exceed memory limits.

```
try:
    li = [1] * (10**10)
except MemoryError as e:
    print(e)
```

Explanation: Attempting to create a list with 10**10 elements requires more memory than available. Python raises a MemoryError.

12. NameError

A NameError occurs when you use a variable or function name that has not been defined.

Example: This example tries to print a variable that was never declared.

```
try:
    print(var)
except NameError as e:
    print(e)
```

Output

name 'var' is not defined

Explanation: Since var was never defined in the code, Python raises a NameError.

13. OSError (and related errors)

Raised when a system-related operation (like file I/O, opening files, or interacting with the OS) fails. In Python 3:

- IOError is just an alias for OSError (they are the same).
- FileNotFoundError is a subclass of OSError, specifically raised when a file or directory does not exist.

Example: This example attempts to open a missing file, which triggers FileNotFoundError (a subclass of OSError).

```
try:
    open("non_existent_file.txt")  # File does not exist
except FileNotFoundError as e:     # More specific
    print("FileNotFoundError caught:", e)
except OSError as e:               # General OS-related error
    print("OSError caught:", e)
```

Output

FileNotFoundError caught: [Errno 2] No such file or directory: 'non_existent_file.txt'

Explanation: The missing file raises FileNotFoundError, which is a subclass of OSError. Since we catch it specifically, the error is displayed clearly.

What is a variable in python? how is it created and assigned a value of particular datatype?

In Python, variables are used to store data that can be referenced and manipulated during program execution. A variable is essentially a name that is assigned to a value.

- Unlike Java and many other languages, Python variables do not require explicit declaration of type.
- The type of the variable is inferred based on the value assigned.

```
x = 5
name = "Samantha"
print(x)
print(name)
```

Output

5

Samantha

In programming, variables are created and assigned values through two primary methods, depending on whether the language is statically typed or dynamically typed.

1. Statically Typed Languages (e.g., Java, C, C++)

In these languages, you must explicitly declare the data type before using a variable. This tells the compiler how much memory to allocate (e.g., 32 bits for an int vs. 64 bits for a double).

Declaration: You specify the type and the variable name.

Example (Java/C): int age;

Assignment: You use the assignment operator (=) to store a value.

Example: age = 25;

Initialization: You can combine both steps into one.

Example: int age = 25;

Constraint: The assigned value must match the declared type; otherwise, the code will fail to compile.

2. Dynamically Typed Languages (e.g., Python, JavaScript)

These languages do not require an explicit data type during declaration. Instead, the type is "inferred" or determined at runtime based on the value assigned to it.

Creation & Assignment: A variable is typically created the moment you assign a value to it.

Example (Python): score = 100 (The language identifies this as an integer)

Example (JavaScript): let name = "Alice"; (Uses keywords like let or const but not a specific type like String)

Flexibility: You can often reassign a variable to a different data type later in the same program.

Example: x = 10 followed by x = "Hello" is valid in Python.

2021

1. **How can we access the middle element in a string?**

> ***Input:*** *str = "GeeksForGeeks"*
> ***Output:*** *o*
> ***Explanation:***
> *The length of the given string is odd.*
> *Therefore, there would be only one middle character, we print that middle character.*

Therefore, there would be only one middle character, we print that middle character.

Approach:

1. Get the string whose middle character is to be found.
2. Calculate the length of the given string.
3. Finding the middle index of the string.
4. Now, print the middle character of the string at index *middle* using function [charAt() in Java](#).

What is the role of the function-index?

5. The `index()` function is used to find the position of a specified element in a sequence such as a list, tuple, or string. It returns the index of the first occurrence of that element.
6. If the element is not found, it raises a ValueError.
7. Example:
8. `l = [10, 20, 30, 40]`
   `print(l.index(30))  # Output: 2`

Explain how Exceptional Handling works in Python.

An Exception is an Unexpected Event, which occurs during the execution of the program. It is also known as a run time error. When that error occurs, Python generates an exception during the execution and that can be handled, which prevents your program from interrupting.

Example: In this code, The system can not divide the number with zero so an exception is raised.

a = 5

b = 0

print(a/b)

Output

```
Traceback (most recent call last):
  File "/home/8a10be6ca075391a8b174e0987a3e7f5.py", line 3, in <module>
    print(a/b)
ZeroDivisionError: division by zero
```

Exception handling with try, except, else, and finally

- Try: This block will test the excepted error to occur
- Except: Here you can handle the error
- Else: If there is no exception then this block will be executed
- Finally: Finally block always gets executed either exception is generated or not

Python Try, Except, else and Finally Syntax

```
try:
    # Some Code....
except:
    # optional block
    # Handling of exception (if required)
else:
    # execute if no exception
finally:
    # Some code .....(always executed)
```

How an element can be deleted from a dictionary?

he pop() method removes the item with the specified key name:

```
thisdict
= {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.pop("model")
print(thisdict)
```

Example

The popitem() method removes the last inserted item (in versions before 3.7, a random item is removed instead):

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.popitem()
print(thisdict)
```

Example

The del keyword removes the item with the specified key name:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
del thisdict["model"]
print(thisdict)
```

Example

The del keyword can also delete the dictionary completely:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
del thisdict
print(thisdict) #this will cause an error because "thisdict" no longer exists.
```

Example

The clear() keyword empties the dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.clear()
print(thisdict)
```

What are diff parameters in a range?

The range() function in Python is used to generate a sequence of integers within a specified range. It is most commonly used in loops to control how many times a block of code runs.

```
for i in range(5):
    print(i, end=" ")
```

Output

0 1 2 3 4

- range(5) generates numbers from 0 to 4
- i takes one value at a time from the generated sequence

Syntax

range(start, stop, step)

Parameters:

- start (optional): Starting number of the sequence (default is 0)
- stop: Number at which the sequence stops (not included)
- step (optional): Difference between consecutive numbers (default is 1)

Return: A range object representing the sequence

Examples

Example 1: This example generates numbers starting from a custom value and ending before another value.

```
for n in range(5, 10):
    print(n, end=" ")
```

Output

5 6 7 8 9

Explanation:

- range(5, 10) starts at 5 and stops before 10
- Numbers increase by the default step of 1

Example 2: This example generates even numbers by skipping values using a custom step size.

```
for v in range(0, 10, 2):
    print(v, end=" ")
```

Output

0 2 4 6 8

Explanation:

- range(0, 10, 2) increases the value by 2 each time
- The step controls how much the number changes per iteration

Example 3: This example demonstrates how range() can be used to generate numbers in reverse order by providing a negative step value.

```
for i in range(10, 0, -2):
    print(i, end=" ")
```

Output

10 8 6 4 2

Explanation:

- range(10, 0, -2) starts from 10 and decreases by 2 each time
- A negative step allows backward iteration through numbers

Discuss polymorphism wrt python with suitable egs

Polymorphism means "many forms". It refers to the ability of an entity (like a function or object) to perform different actions based on the context.

Technically, Polymorphism allows same method, function or operator to behave differently depending on object it is working with. This makes code more flexible and reusable.

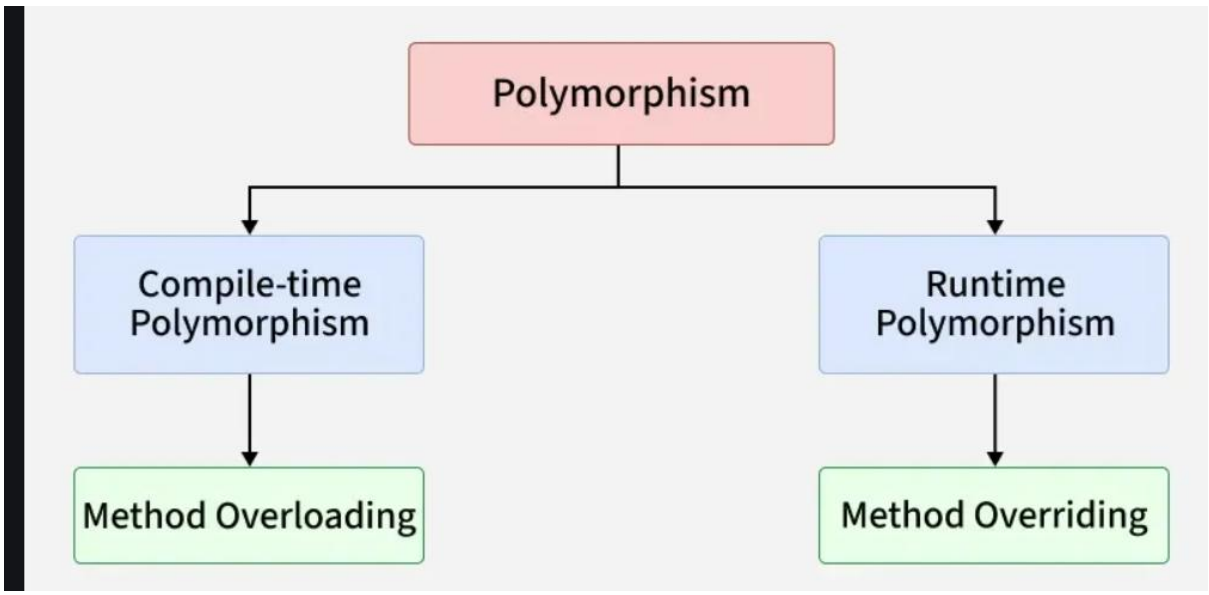
Real Life Example: In a backend payment system, multiple payment options are available such as Credit Card, UPI, NetBanking and Wallet. All payment types use a common method named processPayment() but different implementations:

- Credit Card Payment: validates card, talks to bank API
- UPI Payment: redirects to UPI gateway
- Wallet Payment: checks wallet balance
- NetBanking Payment: redirects to bank login

The method name remains the same, but the action changes based on the payment type.

Types of Polymorphism

Polymorphism refers to ability of the same method or operation to behave differently based on object or context. It mainly includes compile-time and runtime polymorphism.



1. Compile-time Polymorphism

Compile-time polymorphism means deciding which method or operation to run during compilation, usually through method or operator overloading.

Languages like Java or C++ support this. But Python doesn't because it's dynamically typed it resolves method calls at runtime, not during compilation. So, true method overloading isn't supported, though similar behavior can be achieved using default or variable arguments.

Example: This code demonstrates method overloading using default and variable-length arguments. The multiply() method works with different numbers of inputs, mimicking compile-time polymorphism.

```
class Calculator:
    def multiply(self, a=1, b=1, *args):
        result = a * b
        for num in args:
            result *= num
        return result

# Create object
calc = Calculator()
```

```
# Using default arguments
print(calc.multiply())
print(calc.multiply(4))

# Using multiple arguments
print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))
```

Output

1

4

6

24

2. Runtime Polymorphism (Overriding)

Runtime polymorphism means that the behavior of a method is decided while program is running, based on the object calling it. This happens through Method Overriding a child class provides its own version of a method already defined in the parent class.

Example: This code shows runtime polymorphism using method overriding. The sound() method is defined in base class Animal and overridden in Dog and Cat. At runtime, correct method is called based on object's class.

```
class Animal:
    def sound(self):
        return "Some generic sound"

class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    def sound(self):
```

```
    return "Meow"


# Polymorphic behavior
animals = [Dog(), Cat(), Animal()]
for animal in animals:
  print(animal.sound())
```

Output

Bark

Meow

Some generic sound

Explanation: Here, sound method behaves differently depending on whether object is a Dog, Cat or Animal and this decision happens at runtime. This dynamic nature makes Python particularly powerful for runtime polymorphism.

Polymorphism in Built-in Functions

Python's built-in functions like len() and max() are polymorphic they work with different data types and return results based on type of object passed. This showcases it's dynamic nature, where same function name adapts its behavior depending on input.

Example: This code demonstrates polymorphism in built-in functions handling strings, lists, numbers and characters differently while using same function name.

```
print(len("Hello"))  # String length
print(len([1, 2, 3]))  # List length


print(max(1, 3, 2))  # Maximum of integers
print(max("a", "z", "m"))  # Maximum in strings
```

Output

5

3

3

z

Polymorphism in Functions

Polymorphism allows functions to work with different object types as long as they support the required behavior. Using duck typing, it focuses on whether an object has the required methods rather than its type, enabling flexible and reusable code.

Example: This code demonstrates polymorphism using duck typing as perform_task() function works with different object types (Pen and Eraser), as long as they have a .use() method showing flexible and reusable function design.

```
class Pen:
    def use(self):
        return "Writing"


class Eraser:
    def use(self):
        return "Erasing"


def perform_task(tool):
    print(tool.use())


perform_task(Pen())
perform_task(Eraser())
```

Output

Writing

Erasing

Explain how the read function works in Python. Explain the context of seek() func with respect to read

The read() method returns the specified number of bytes from the file. Default is -1 which means the whole file.

Syntax

file.read()

Parameter Values

Parameter	Description
size	Optional. The number of bytes to return. Default -1, whichmeans the whole file.

More examples

Example

Read the content of the file "demofile.txt":

```
f = open("demofile.txt", "r")
print(f.read(33))
```

The seek() function in file handling (specifically in Python) plays a crucial role in random access reading, allowing you to move the file cursor to a specific position rather than reading sequentially from the beginning. It determines exactly where the next read() operation will begin, enabling efficient navigation to specific data points within large files without loading the entire file into memory

For example, if you want to skip the first 10 characters while reading a file, you can do:

```
f = open("demo.txt", "r")

f.seek(10)

print(f.read())

f.close()
```

This will move the file cursor to position 10 and begin reading from there.

Syntax:

file.seek(offset, from_what)

- offset: Number of bytes to move the cursor.
- from_what: (optional) Reference point to start from:
    - 0: sets the reference point at the beginning of the file
    - 1: sets the reference point at the current file position

explain different types of operators in Python with example

In Python programming, Operators in general are used to perform operations on values and variables.

- Operators: Special symbols like -, + , * , /, etc.
- Operands: Value on which the operator is applied.

Types of Operators in Python

Operators in Python

Type	Operators
Arithmetic Operators	+, -, *, /, %
Relational Operators	< , <= , > , >=, ==, !=
Logical Operators	AND, OR, NOT
Bitwise operator	&, |, ^, ~, <<, >>
Assignment Operator	= , += , -= , *= , %=

Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication and division.

In Python 3.x the result of division is a floating-point while in Python 2.x division of 2 integers was an integer. To obtain an integer result in Python 3.x floored (// integer) is used.

```
# Variables

a = 15

b = 4


# Addition

print("Addition:", a + b)


# Subtraction

print("Subtraction:", a - b)


# Multiplication

print("Multiplication:", a * b)
```

```
# Division
print("Division:", a / b)

# Floor Division
print("Floor Division:", a // b)

# Modulus
print("Modulus:", a % b)

# Exponentiation
print("Exponentiation:", a ** b)
```

Output

Addition: 19

Subtraction: 11

Multiplication: 60

Division: 3.75

Floor Division: 3

Modulus: 3

Exponentiation: 50625

Note: Refer to Differences between / and //.

Comparison Operators

Comparison (or Relational) operators compares values. It either returns True or False according to the condition.

```
a = 13
b = 33

print(a > b)
print(a < b)
```

```
print(a == b)
print(a != b)
print(a >= b)
print(a <= b)
```

Output

```
False
True
False
True
False
True
```

Logical Operators

Logical operators perform Logical AND, Logical OR and Logical NOT operations. It is used to combine conditional statements.

The precedence of Logical Operators in Python is as follows:

1. Logical not
2. logical and
3. logical or

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

Output

```
False
True
False
```

Bitwise Operators

Bitwise operators act on bits and perform bit-by-bit operations. These are used to operate on binary numbers.

Bitwise Operators in Python are as follows:

1. Bitwise NOT
2. Bitwise Shift
3. Bitwise AND
4. Bitwise XOR
5. Bitwise OR

```
a = 10
b = 4

print(a & b)
print(a | b)
print(~a)
print(a ^ b)
print(a >> 2)
print(a << 2)
```

Output

```
0
14
-11
14
2
40
```

Assignment Operators

Assignment operators are used to assign values to the variables. This operator is used to assign the value of the right side of the expression to the left side operand. Example:

```
a = 10
b = a
```

```
print(b)
b += a
print(b)
b -= a
print(b)
b *= a
print(b)
b <<= a
print(b)
```

Output

```
10
20
10
100
102400
```

Identity Operators

"is" and "is not" are the identity operators and both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

is True if the operands are identical
is not True if the operands are not identical

```
a = 10
b = 20
c = a

print(a is not b)
print(a is c)
```

Output

```
True
```

True

Membership Operators

"in" and "not in" are the membership operators that are used to test whether a value or variable is in a sequence.

in True if value is found in the sequence
not in True if value is not found in the sequence

```
x = 24

y = 20

list = [10, 20, 30, 40, 50]


if (x not in list):

   print("x is NOT present in given list")

else:

   print("x is present in given list")


if (y in list):

   print("y is present in given list")

else:

   print("y is NOT present in given list")
```

Output

x is NOT present in given list

y is present in given list

Ternary Operator

Ternary operators also known as conditional expressions are operators that evaluate something based on a condition being true or false. It was added to Python in version 2.5.

It simply allows testing a condition in a single line replacing the multiline if-else, making the code compact.

Syntax : [on_true] if [expression] else [on_false]

```
a, b = 10, 20

min = a if a < b else b
```

```
print(min)
```

Output

10

what is the utility of append function explain with an example

append() method in Python is used to add a single item to the end of list. This method modifies the original list and does not return a new list. Let's look at an example to better understand this.

```
a = [2, 5, 6, 7]

# Use append() to add the element 8 to the end of the list
a.append(8)
print(a)
```

Output

[2, 5, 6, 7, 8]

Explanation: append(8) adds 8 to the end of the list a, modifying it in place.

Syntax of append() method

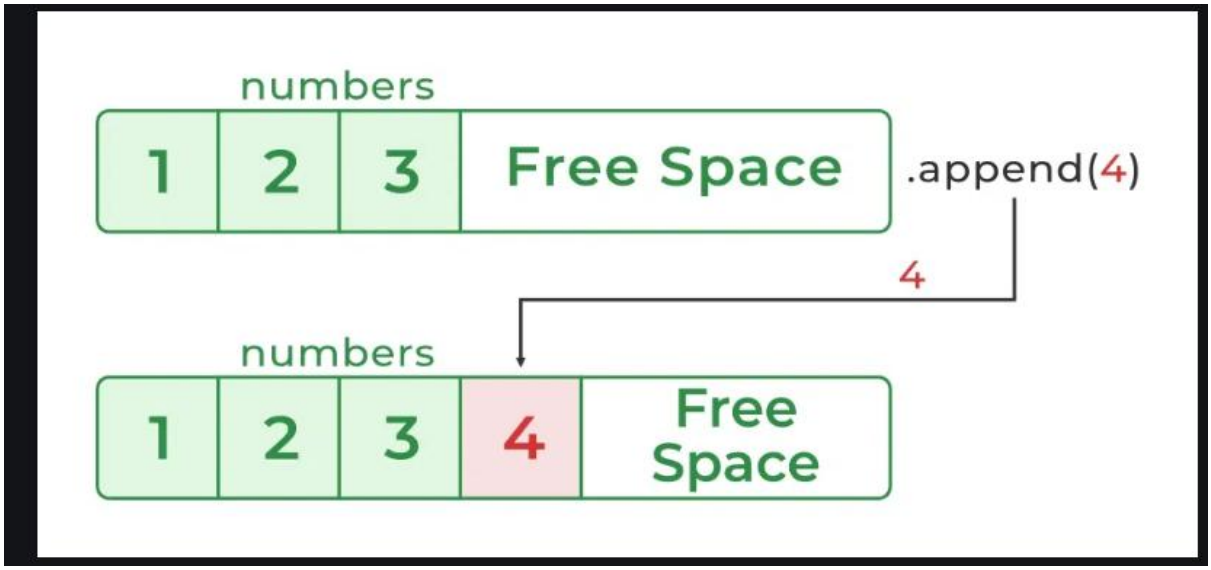
list.append(element)

Parameter:

- element: The item to be appended to the list. This can be of any data type (integer, string, list, object, etc.). This parameter is mandatory, and omitting it will cause an error.

Return:

- append() does not return any value. It modifies the original list in place.



Examples of append() Method

Here are some examples and use-cases of list append() function in Python.

1. Appending Elements of Different Types

The append() method allows adding elements of different data types (integers, strings, lists or objects) to a list. Python lists are heterogeneous meaning they can hold a mix of data types.

```
a = [1, "hello", 3.14]


a.append(True)

print(a)
```

Output

```
[1, 'hello', 3.14, True]
```

Explanation: In this list "a" contains elements of different data types (integer, string, float) and append(True) adds a boolean True to the end of the list.

2. Appending List to a List

When appending one list to another, the entire list is added as a single element, creating a nested list.

```
a = [1, 2, 3]


a.append([4, 5])
```

print(a)

Output

[1, 2, 3, [4, 5]]

Explanation: append() method adds the list [4, 5] as a single element to the end of the list a, resulting in a nested list.

3. Appending Using a Loop

Appending multiple elements using a loop:

a = []

for i in range(5):

  a.append(i)

print(a)

Output

[0, 1, 2, 3, 4]

What are the usages of '\t' and '*' operators on string in Python?When the value error occurs in python?

In Python, the backslash \t and the star * operator serve distinct purposes for string formatting and manipulation. The \t is an escape sequence for inserting horizontal space, while the * operator acts as a repetition operator for strings.

1. Backslash \t (Tab Character)

The \t is a special character sequence used within string literals to represent a horizontal tab

Use Cases:

- Formatting/Alignment: It adds horizontal space, often used to align text into columns without using external libraries.
- Whitespace Insertion: It inserts a "tab" space inside a string

- Example:
- python
- ```
  print("Name\tAge\tLocation")
  print("John\t25\tNew York")
  # Output:
  # Name    Age    Location
  ```
  ```

- # John 25 New York
- 2. Star Operator `*` (Repetition)
- When used with strings, the star `*` operator (also called the repetition operator or multiplication operator) repeats a string a given number of times

- Syntax: string * integer or integer * string
- Use Cases:
  - Repeating Strings: Creating strings with repeated characters or patterns.
  - Creating Visual Dividers: Generating lines, separators, or patterns in terminal output

separator = "-" * 10

print(separator)

Output: ----------

word = "Echo"

print(word * 3)

Output: EchoEchoEcho

A ValueError occurs if you send a string into a function that requires a number:

x = float("hello")

Definition and Usage

The ValueError exception occurs if a function receives a value of wrong type.

You can handle the ValueError in a try...except statement, see the example below.

Exception Handling

Example

Handling the ValueError in a try...except statement:

```
try:
  x = float("hello")
except ValueError:
```

```
  print("The value has wrong format")
except:
  print("Something else went wrong")
```